

1. ABSTRACT

This project deals with the various functioning in college management process. The main idea is to implement a proper process to system. In our existing system contains a many operations Students' registration, fees, Marks, exam records, Sports and Events of the student etc. All these activity takeout systematically by administrator. In our model, it deals with the operations in system. For example, when admin add all the forms in this system stored in the database. And display the details of student is perform by retrieving information from database table.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION:

As the college and information services community seeks to cope with new patterns of information provision, new technology, and changing financial circumstances, it is critical to gain new thinking across the profession. The innovative theory, and best organizational practice are all presented in the College Management System. The college management system provides knowledge about all aspects related to detailed information for students in order to follow them effectively. The college management system is one of the most tedious processes, which includes the regular updating of many files and records. When new students register, the administration has to issue a unique code for that student. For this, they have to keep track of many records like student details, address, phone number etc. and keep many records. Doing this manually takes a lot of time and effort. Hence, this can be facilitated by automation.

2.2 EXISTING SYSTEM:

The college is an ideal candidate for an IT solution, due to the vast amounts of data that it has to deal with on a daily basis. If the administration wants to search for a record of the student, they have to search for all records. If the administration wants to add a new student to the database, they must search for the subject or branch the student belongs to. If students want to know the financial situation or what is related to the fees, they must in the manual system come to the college to find out the status. The whole process is now controlled automatically. This contributes to the fact that records of inquiries from students do not need to be kept on paper.

ADVANTAGES:

An exit system in a College Management System (CMS) using Python and MySQL, while providing a means for users to safely log out or close the application, can have certain disadvantages. One potential issue is the risk of data loss or corruption if the system is not properly designed to handle session termination or if there is a failure during the exit process. Furthermore, excessive reliance on the exit system could result in performance bottlenecks if not optimized, especially when the system has multiple users accessing the database at the same time. In the worst-case scenario, a poorly implemented exit system could cause security vulnerabilities, such as unauthorized users gaining access due to improperly closed sessions or

unencrypted connections. Therefore, careful planning and testing of the exit mechanism are crucial to ensure that the system remains stable, secure, and reliable.

DISADVANTAGES:

Maintaining the data in excel sheets and files is very hard to remember the file names in which the required data is feed. No easy access to the required queries. Data redundancy, inconsistency, a lot of human work need to be done to analyses the details present in the excel sheets. It leads to wastage of time. No timely response to the end-users.

2.3 PROPOSED SYSTEM:

The proposed system is the one that is going to be automated so that it is easy to retrieve the responses from the system fastly and updating the details once the response or services are provided to the end-users upon their request without any difficulty and saves time.

ADVANTAGE

- Computerization of the details of the students.
- Maintenance time and cost are greatly reduced.
- Accurate information can be generated easily and quickly at different levels.

DISADVANTAGE

One disadvantage of a proposed college management system implemented in Python with MySQL is its dependency on a stable internet connection for the MySQL database. If the connection is lost, the system may experience downtime or fail to retrieve or update critical data, disrupting college operations. Additionally, while Python is versatile, its performance may not be as optimal as other languages like Java or C++ when handling large-scale data, especially in systems with thousands of users and extensive data processing. Another concern is the potential for security vulnerabilities in the database if proper encryption and access control measures are not implemented, which could expose sensitive student and staff information. Lastly, integrating the system with other existing software or legacy systems in the college may require significant development effort, adding complexity and increasing the risk of integration issues.

3.SYSTEM SPECIFICATION

3.1 HARDWARE REQUIREMENTS:

System	:	Pentium IV 2.4 GHz.
Hard Disk	:	240 GB.
Monitor	:	14' Colour Monitor.
Mouse	:	Optical Mouse.
Ram	:	8 GB.

3.2 SOFTWARE REQUIREMENTS:

Operating system	:	Windows 11
Coding Language	:	Python
IDELE	:	IDLE (Python)

4. SYSTEM DESIGN

4.1 INTRODUCTION

Design is the process of translating requirements defined during analysis into several designs activities for system requirements. The designer select requirements needed to implement the system in this phase; the design of the database also takes place. After identifying the problem, limitations are opportunities to improve the efficiency system. A detail design of the proposed system is done. In database design several objectives are considered such as,

- Controlled Redundancy
- Data Independence
- More Information at low cost
- Accuracy and Integrity
- Recovery and Failure
- Security
- Performance

SOFTWARE DESCRIPTION:

PYTHON

Software in Python refers to a collection of code that performs specific tasks, designed to run on Python's interpreter. It typically involves writing instructions to be executed by the Python runtime environment. Python software can vary from simple scripts to large, complex systems.

Components of Python Software

1. Source Code: The core of Python software is its source code, written in Python language. The code typically consists of functions, classes, methods, and modules that define the behavior of the software. It might include algorithms, data structures, logic to interact with other systems, and more.

2. Libraries and Frameworks: Python has an extensive collection of libraries and frameworks. These are pre-written codebases that provide common functionality. Libraries like NumPy (for scientific computing), Pandas (for data analysis), and TensorFlow (for machine learning) are examples. Frameworks such as Django or Flask provide structures for building web applications

3. Modules: A module is a single file containing Python code, which can be imported into other Python programs. These can be built-in modules or third-party libraries. A Python software package typically organizes functionality into multiple modules for ease of maintenance and scalability.

4. Data Structures: Python software uses a variety of built-in data structures like lists, dictionaries, sets, and tuples. More complex data structures, like queues, stacks, linked lists, trees, and graphs, may also be implemented based on the needs of the software.

5. Control Flow: Like other programming languages, Python software utilizes control flow statements such as conditionals (if-else), loops (for, while), and function calls to dictate the flow of execution.

6. Object-Oriented Programming (OOP): Python supports object-oriented programming principles, which help in organizing code into classes and objects. This promotes code reuse, modularity, and abstraction. Concepts like inheritance, polymorphism, encapsulation, and abstraction are central to designing robust Python software.

7. Error Handling: Error handling in Python is done through exceptions. Python software includes mechanisms to catch and handle errors, preventing crashes and providing informative feedback when something goes wrong.

8. Input and Output (I/O): Python software often interacts with external resources like files, databases, or network sockets. Input and output operations are crucial in most applications, whether reading from or writing to files, interacting with users via command-line input, or making HTTP requests.

9. Concurrency and Parallelism: For performance optimization, especially in I/O-bound or CPU-bound tasks, Python software can leverage concurrency (using threads or asynchronous programming) or parallelism (using multiple processors) to perform tasks simultaneously or in a non-blocking manner.

10. Testing and Debugging: Python software often includes test suites written using frameworks like unittest or pytest to ensure that the code works as expected. Debugging tools, like Python's built-in pdb, are often used to diagnose issues during development.

11. Documentation: High-quality Python software includes comprehensive documentation that describes its functionality, installation steps, usage, and dependencies. Python developers often use docstrings to document functions, classes, and methods inline with the code.

12. Dependencies: Many Python applications depend on external packages, which are typically managed using tools like pip or conda. The list of dependencies is commonly stored in a requirements.txt or a Pipfile.

Python Software Development Process

1. Planning and Design: Before writing any code, software developers plan the features, functionalities, and architecture. This might involve creating flowcharts, diagrams, and specifications that describe how the software will behave.

2. Development: The actual coding happens here. It might involve implementing algorithms, creating functions, setting up classes, and using libraries to build out the software.

3. Testing: After development, the software is thoroughly tested to catch any bugs or inconsistencies. Unit tests, integration tests, and system tests are common. Python has various testing frameworks that make this process smoother.

4. Deployment: Once the software is developed and tested, it's deployed. This could involve packaging the Python code for distribution, creating an executable, or deploying the software on a server or cloud platform.

5. Maintenance and Updates: Post-deployment, software requires ongoing maintenance. This involves bug fixes, security patches, and sometimes adding new features based on user feedback.

PYTHON HISTORY

Python was created by Guido van Rossum and first released in 1991. It was designed to be easy to read and write, emphasizing code readability and simplicity. Python's development was influenced by several programming languages, including ABC, a teaching language van Rossum had worked with at the Centrum Wiskunde & Informatica (CWI) in the Netherlands.

Over the years, Python grew in popularity due to its versatility, extensive standard library, and supportive community. It was adopted by various fields, including web development, data

science, and automation. Python's clean syntax made it a popular choice for beginners and experts alike. Major versions like Python 2 and Python 3 were released, with Python 3 introducing improvements that were not backward compatible with Python 2, leading to a lengthy transition period.

Python's design philosophy, encapsulated by the "Zen of Python," emphasizes simplicity and readability. Its open-source nature, along with an ever-expanding ecosystem of libraries and frameworks, has cemented Python as one of the most widely used programming languages globally.

4.2 TABLE DESIGN

Admin Table

Column	Data Type	Length
Name	Varchar	50
Contact	Varchar	50
ID	Varchar	50
Password	Varchar	50

Depts Table

Column	Data Type	Length
Sno	Int	11
Degree	Varchar	50
Department	Varchar	50
Description	Varchar	50

Event Table

Column	Data Type	Length
EDate	Int	11
Event	varchar	50
Place	varchar	50
Description	varchar	50

Exam Table

Column	Data Type	Length
EDate	varchar	50
Exam	varchar	50
Subject	varchar	50
Semester	varchar	5000

Fee Table

Column	Data Type	Length
Sno	Int	11
Degree	varchar	50
Department	varchar	50
TutionFee	varchar	50
LabFee	varchar	50
SFee	varchar	50
Total	varchar	50

Labs Table

Column	Data Type	Length
Sno	Int	11
Labname	varchar	50
Department	varchar	50
Description	varchar	5000

Marks Table

Column	Data Type	Length
Regno	varchar	50
Name	varchar	50
S1	varchar	50
S2	varchar	50
S3	varchar	50
S4	varchar	50
S5	varchar	50
S6	varchar	50
Total	varchar	50

Sports Table

Column	Data Type	Length
Sports	varchar	50
Age	Varchar	50
Height	Varchar	50
Weight	Varchar	50
Description	Varchar	5000

Staff Table

Column	Data Type	Length
Name	varchar	50
Qualification	Varchar	50
Email	Varchar	50
Mobile	Varchar	50
Salary	Varchar	50
Gender	Varchar	50

Student Table

Column	Data Type	Length
FName	varchar	50
LName	Varchar	50
Email	Varchar	50
Mobile	Varchar	50
DOB	Varchar	50
Gender	Varchar	50
SClass	Varchar	50
Regno	Varchar	50

Subjects Table

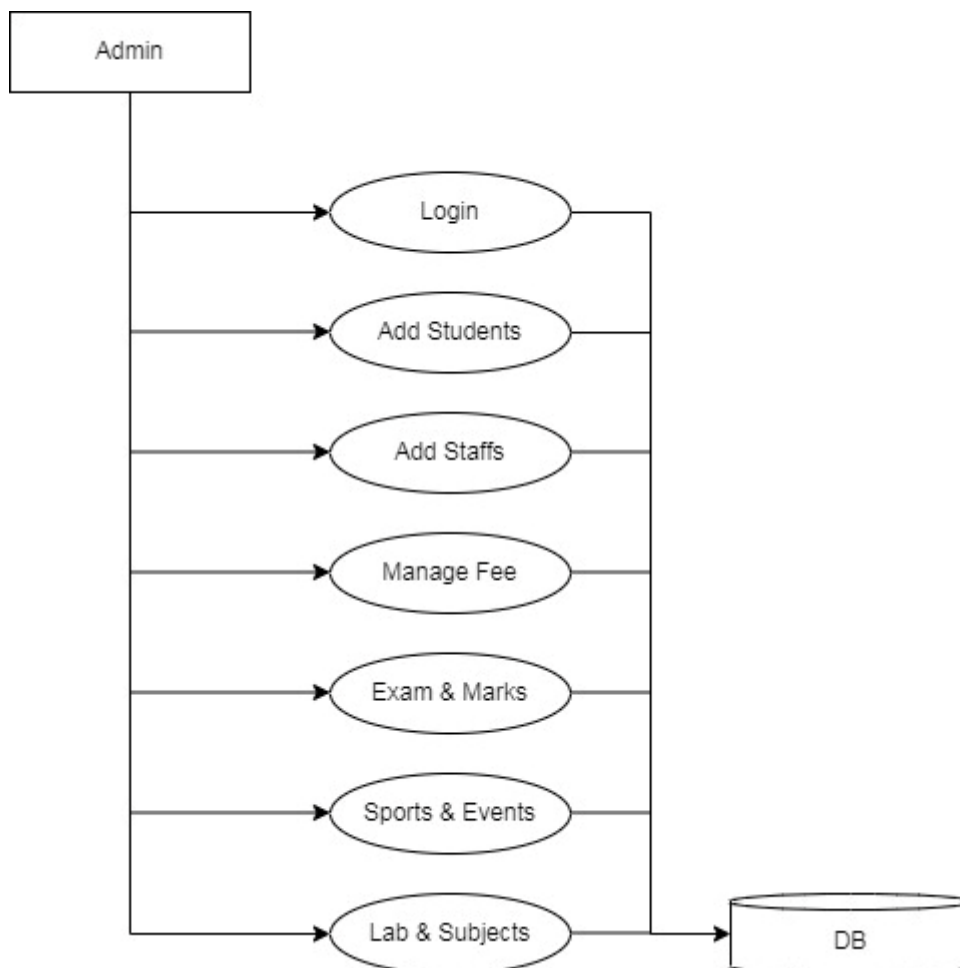
Column	Data Type	Length
Sno	varchar	50
Subject	Varchar	50
Department	Varchar	50
Staff	Varchar	50

4.3 DATA FLOW DIAGRAM:

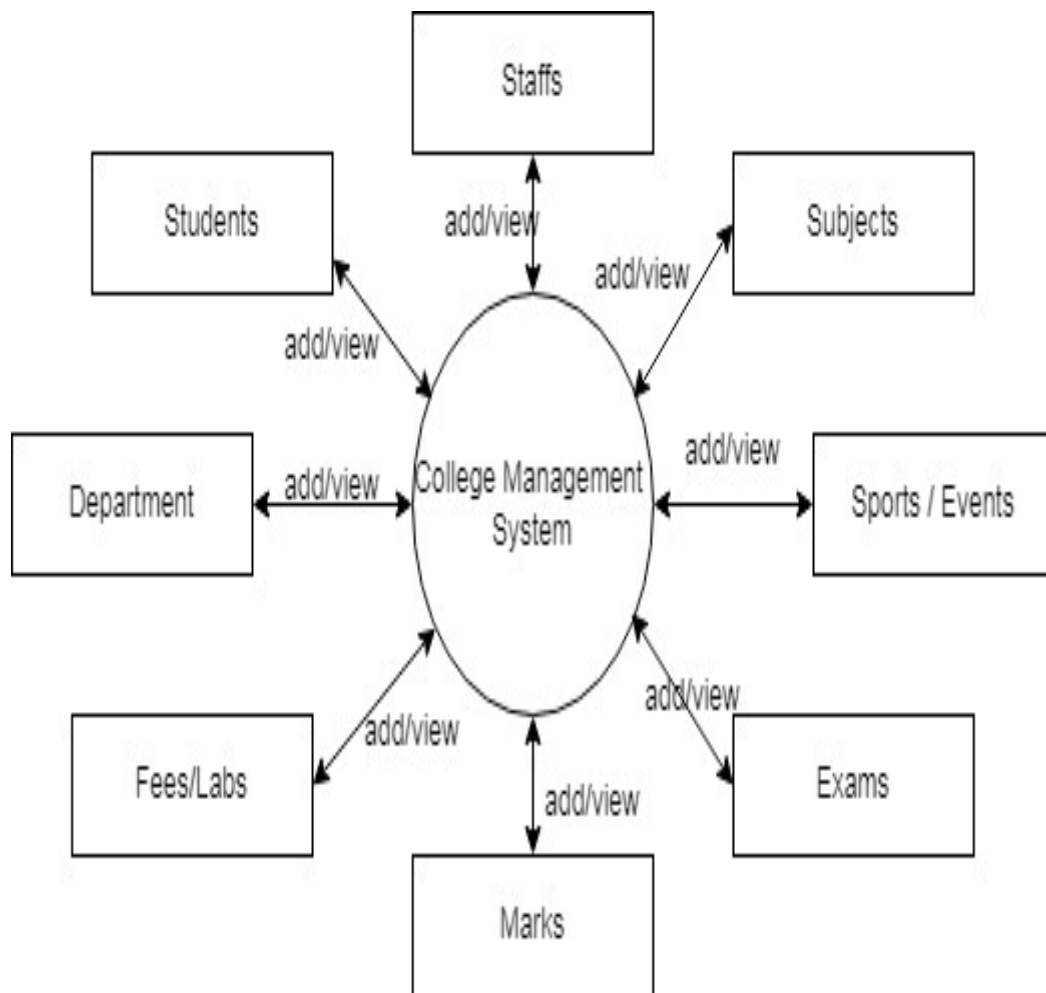
Level 0



Level 1



4.4 CONTEXT DIAGRAM:



5.SYSTEM TESTING AND IMPLEMENTATION

5.1 TESTING:

Testing is a process of executing a program with the intent of finding an error. Testing is a crucial element of software quality assurance and presents ultimate review of specification, design and coding. System Testing is an important phase. Testing represents an interesting anomaly for the software. Thus a series of testing are performed for the proposed system before the system is ready for user acceptance testing.

A good test case is one that has a high probability of finding an as undiscovered error. A successful test is one that uncovers an as undiscovered error.

- Testing is a process of executing a program with the intent of finding an error Exhaustive testing is not possible
- To be most effective testing should be conducted by a independent third party.
- The primary objective for test case design is to derive a set of tests that has the highest livelihood for uncovering defects in software.
- To accomplish this objective two different categories of test case design techniques are used. They are

Unit Testing:

Unit testing is essential for the verification of the code produced during the coding phase and hence the goal is to test the internal logic of the modules. Using the detailed design description as a guide, important paths are tested to uncover errors within the boundary of the modules. These tests were carried out during the programming stage itself. All units of Vienna SQL were successfully tested.

Integration Testing:

Integration testing focuses on unit tested modules and build the program structure that is dictated by the design phase.

System Testing:

System testing tests the integration of each module in the system. It also tests to find discrepancies between the system and it is original objective, current specification and system documentation. The primary concern is the compatibility of individual modules. Entire system

is working properly or not will be tested here, and specified path ODBC connection will correct or not, and giving output or not are tested here these verifications and validations are done by giving input values to the system and by comparing with expected output. Top-down testing implementing here.

Acceptance Testing:

This testing is done to verify the readiness of the system for the implementation. Acceptance testing begins when the system is complete. Its purpose is to provide the end user with the confidence that the system is ready for use. It involves planning and execution of functional tests, performance tests and stress tests in order to demonstrate that the implemented system satisfies its requirements.

Tools to special importance during acceptance testing include:

- Test coverage Analyzer – records the control paths followed for each test case.
- Timing Analyzer – also called a profiler, reports the time spent in various regions of the code are areas to concentrate on to improve system performance.
- Coding standards – static analyzers and standard checkers are used to inspect code for deviations from standards and guidelines.

Test Cases:

Test cases are derived to ensure that all statements in the program have been executed at least once during testing and that all logical conditions have been executed. Using White-Box testing methods, the software engineer can drive test cases that

- Guarantee that logical decisions on their true and false sides.
- Exercise all logical decisions on their true and false sides.
- Execute all loops at their boundaries and within their operational bounds.
- Exercise internal data structure to assure their validity.
- The test case specification for system testing has to be submitted for review before system testing commences.

5.2 SYSTEM IMPLEMENTATION

- Login
- Student Module
- Staff Module
- Mark Module
- Subject Module
- Exam Module
- Department Module
- Sports and Events Module
- Laboratory Module
- Fee module

Module Description

Login

This module is used to authentication admin to provide ID and Password for Login, After Login Successful the system opens the application.

Student Module

The admin can add and manage the student details. The system will Store student Records in this system using Database.

Staff Module

This module is used to add staffs details and view staffs details. This detail consists of staff salary and contact details.

Mark Module

Mark module is to store the exam result of every student, it contains RegNo, Name and Marks

Subject Module

This module is used to store department wise Subjects and Staff allocation for all the subjects

Exam Module

In this module was storing the examination details like exam date, subject and other details.

Department Module

This module is to clustering the department using degree, these details has stored by the admin using various Programs

Sports and Events Module

The admin can add the sports details to this system and view these details it contains all Eligibility criteria for every sports. And also admin can store the event details like workshop, Seminar etc.

Laboratory Module

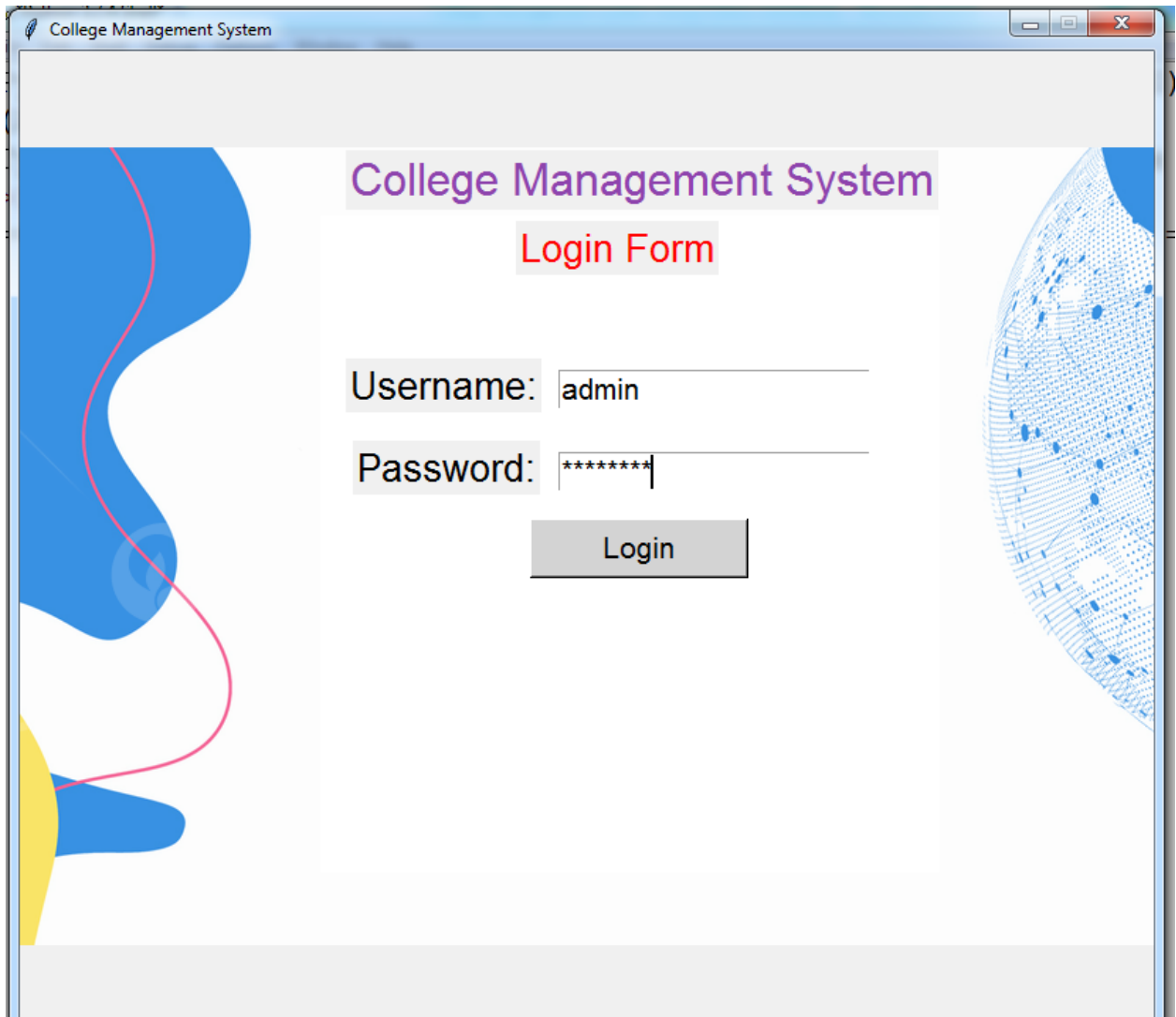
The laboratory module is to store the details of lab name, Equipments and departments in this system.

Fee module

This module is used to Store and View the Fee Structure like tuition fee, lab fee and other fee.

6. FORMS

Login



The image shows a screenshot of a web application window titled "College Management System". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. The main content area has a light gray background with decorative blue and yellow abstract shapes on the left and right sides. In the center, there is a white rectangular box containing the login form. The form has a title "College Management System" in purple and "Login Form" in red. Below the title, there are two input fields: "Username:" with the text "admin" and "Password:" with masked characters "*****". A "Login" button is positioned below the password field.

College Management System

College Management System

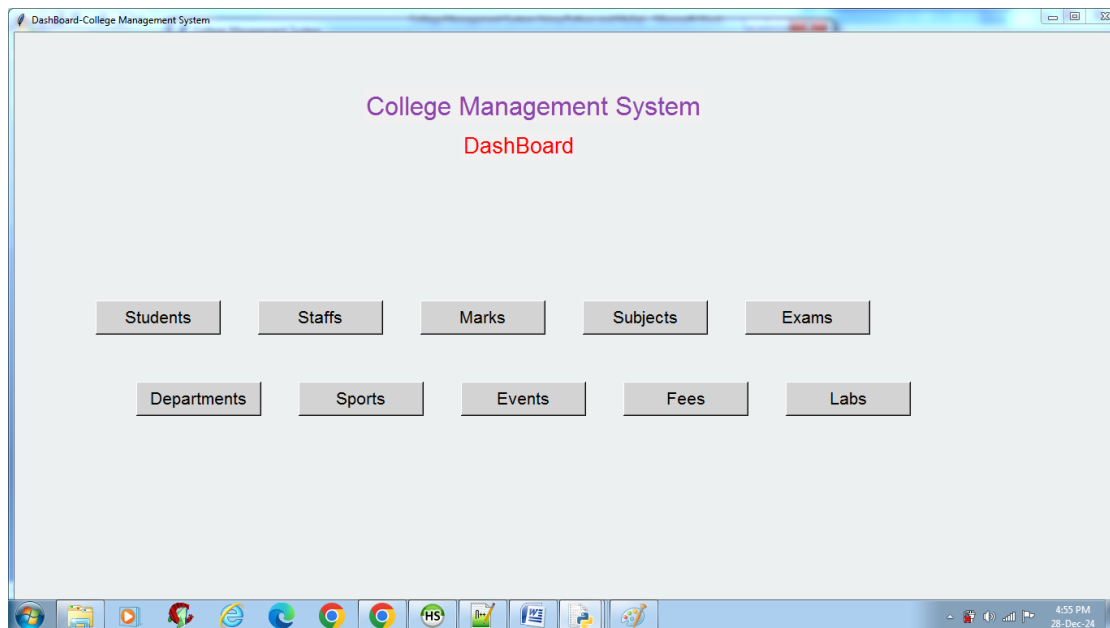
Login Form

Username: admin

Password: *****

Login

Dashboard



Students

The screenshot shows the 'Student Details' form within the 'College Management System'. The title 'College Management System' is in purple, and 'Student Details' is in red below it. A 'View' button is in the top right corner. The form contains the following fields and values:

First Name:	Samir	Last Name:	A
EmailID:	shamir@gmail.com	Mobile:	9876543210
DOB:	10-12-2007	Gender:	☒ Male ☐ Female
Class:	BE-CSE	Reg No:	10206789

A 'Submit' button is located at the bottom center of the form. The Windows taskbar at the bottom shows various icons and the system clock indicating 4:56 PM on 28-Dec-24.

Staff

The screenshot shows a web browser window titled "DashBoard-College Management System". Inside, a sub-window titled "Staffs-College Management System" contains the "Staff Details" form. The form has a "View" button in the top right. The title "College Management System" is in purple, and "Staff Details" is in red. The form fields are: Name (text input), Qualification (text input), EmailID (text input), Mobile (text input), Salary (text input), and Gender (radio buttons for Male and Female, with Female selected). A "Submit" button is at the bottom center. The Windows taskbar at the bottom shows various icons and the system clock reading 4:57 PM on 28-Dec-24.

DashBoard-College Management System

Staffs-College Management System

View

College Management System

Staff Details

Name: Qualification:

EmailID: Mobile:

Salary: Gender: ☐ Male ☒ Female

Submit

Marks

The screenshot shows a web browser window titled "DashBoard-College Management System". Inside, a sub-window titled "Marks-College Management System" contains the "Marks Details" form. The form has a "View" button in the top right. The title "College Management System" is in purple, and "Marks Details" is in red. The form fields are: Reg No (text input), Name (text input), Subject 1 (text input), Subject 2 (text input), Subject 3 (text input), Subject 4 (text input), Subject 5 (text input), and Subject 6 (text input). A "Submit" button is at the bottom center. The Windows taskbar at the bottom shows various icons and the system clock reading 4:58 PM on 28-Dec-24.

DashBoard-College Management System

Marks-College Management System

View

College Management System

Marks Details

Reg No: Name:

Subject 1: Subject 2:

Subject 3: Subject 4:

Subject 5: Subject 6:

Submit

Subjects

DashBoard-College Management System

Subjects-College Management System

View

College Management System

Subjects Details

Subject:

Department:

Staff:

Submit

4:59 PM 28-Dec-24

Exams

DashBoard-College Management System

Exams-College Management System

View

College Management System

Exam Date & Details

Date: 12/28/24

Department:

Subject:

Semester:

Submit

1
2
3
4
5
6

5:00 PM 28-Dec-24

Departments

The screenshot shows a web browser window titled "Dashboard-College Management System". The main content area is titled "Department-College Management System". In the top right corner, there is a "View" button. The form is titled "College Management System" in purple, with "Department Details" in red below it. The form contains three input fields: "Degree:" with a single-line text box, "Department:" with a single-line text box, and "Description:" with a multi-line text box. A "Submit" button is located at the bottom center of the form. The Windows taskbar at the bottom shows various application icons and the system clock indicating 5:00 PM on 28-Dec-24.

Dashboard-College Management System

Department-College Management System

View

College Management System

Department Details

Degree:

Department:

Description:

Submit

5:00 PM
28-Dec-24

Sports

The screenshot shows a web browser window titled "Dashboard-College Management System". The main content area is titled "Sports-College Management System". In the top right corner, there is a "View" button. The form is titled "College Management System" in purple, with "Sports Details" in red below it. The form contains five input fields: "Sports:" with a single-line text box, "Age:" with a single-line text box, "Hight:" with a single-line text box, "Weight:" with a single-line text box, and "Description:" with a multi-line text box. A "Submit" button is located at the bottom center of the form. The Windows taskbar at the bottom shows various application icons and the system clock indicating 5:02 PM on 28-Dec-24.

Dashboard-College Management System

Sports-College Management System

View

College Management System

Sports Details

Sports:

Age:

Hight:

Weight:

Description:

Submit

5:02 PM
28-Dec-24

Events

DashBoard-College Management System

Events-College Management System

College Management System

Event Date & Place

Date: 12/28/24

EventName:

Place:

Description:

Submit

View

5:03 PM 28-Dec-24

Fee Details

DashBoard-College Management System

Fees Details-College Management System

College Management System

Fee Details / Year

Degree:

Department:

TutionFee:

LabFee:

Stationary:

Submit

View

5:03 PM 28-Dec-24

Labs

The screenshot shows a web browser window with the title 'Dashboard-College Management System'. The address bar shows 'College Management System: http://localhost:8080/CollegeManagementSystem'. The browser tab is labeled 'Laboratory-College Management System'. The main content area displays the 'College Management System' title in purple, followed by 'Laboratory-Details' in red. A 'View' button is in the top right corner. The form contains three input fields: 'LabName:', 'Department:', and 'Description:'. A 'Submit' button is at the bottom of the form. The Windows taskbar at the bottom shows various application icons and the system clock indicating 5:04 PM on 28-Dec-24.

Dashboard-College Management System

College Management System: http://localhost:8080/CollegeManagementSystem

Laboratory-College Management System

View

College Management System

Laboratory-Details

LabName:

Department:

Description:

Submit

5:04 PM
28-Dec-24

7. SOURCE CODE

Login

```
from tkinter import *
from tkinter import messagebox, Label
import mysql.connector
from PIL import ImageTk, Image
import os

mydb=mysql.connector.connect(
    host="localhost",
    user="root",
    passwd="good",
    database="mydbs"
)
mycursor=mydb.cursor(buffered=True)
#mycursor.execute('SELECT * FROM AI_login.AI_login')

login = Tk()
login.title('College Management System')
login.geometry('800x700')
img = ImageTk.PhotoImage(Image.open("back.jpg"))
panel = Label(login, image = img)
panel.pack(side = "bottom", fill = "both", expand = "yes")
login.resizable(0, 0)

def connect_database():
    # modify your SELECT query to select the correct columns
    mycursor.execute('SELECT Name, Address FROM customers')
    username = usernameEntry1.get()
    password = passwordEntry2.get()
    check_login = f"SELECT Name FROM customers WHERE Name = '{username}'"
    check_password = f"SELECT Address FROM customers WHERE Address = '{password}'"
```

```

mycursor.execute(check_login)
username_result = mycursor.fetchall()
mycursor.execute(check_password)
password_result = mycursor.fetchall()
usernameU = username_result
passwordP = password_result
print(username)
print(password)
if usernameU and passwordP and username == usernameU[0][0] and password ==
passwordP[0][0]:
    #messagebox.showinfo(title="", message='Login successfully')
    #login.destroy()
    import Dashboard
else:
    messagebox.showerror(title='Error', message='Error, username/password is incorrect')

heading : Label = Label(login, text='College Management System', font=('Arial', 24),
fg='#8e44ad')
heading.place(x=230, y=70)
heading : Label = Label(login, text='Login Form', font=('Arial', 20), fg='red')

heading.place(x=350, y=120)

heading=Label(login, text='Username:', font=('Arial', 20))
heading.place(x=230, y=217)

usernameEntry1=Entry(login, font=('Arial', 16), width=18)
usernameEntry1.place(x=380,y=225)

heading=Label(login, text='Password:', font=('Arial', 20))
heading.place(x=235, y=275)

passwordEntry2=Entry(login, font=('Arial', 16), width=18)
passwordEntry2.place(x=380,y=283)

```

```
passwordEntry2.configure(show='*') loginButton=Button(login, text='Login', font=('Arial',
16),bg='light grey', width=12, height=1, command=connect_database)
loginButton.place(x=360, y=330)
login.mainloop()
```

Dashboard

```
from tkinter import *
ws = Tk()
ws.geometry('1500x2000')
ws.title('DashBoard-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)
def view_students():
    import Students
def view_staffs():
    import Staffs
def view_marks():
    import Marks
def view_subjects():
    import Subjects
def view_exams():
    import Exams
def view_depts():
    import Depts
def view_sports():
    import Sports
def view_events():
    import Events
def view_fees():
    import Fees
def view_labs():
    import Labs
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
```

```
heading : Label = Label(ws, text='DashBoard', font=('Arial', 20), fg='red')
```

```
heading.place(x=550, y=120)
```

```
students=Button(ws, text='Students', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_students)
```

```
students.place(x=100, y=330)
```

```
staffs=Button(ws, text='Staffs', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_staffs)
```

```
staffs.place(x=300, y=330)
```

```
marks=Button(ws, text='Marks', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_marks)
```

```
marks.place(x=500, y=330)
```

```
subjects=Button(ws, text='Subjects', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_subjects)
```

```
subjects.place(x=700, y=330)
```

```
exams=Button(ws, text='Exams', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_exams)
```

```
exams.place(x=900, y=330)
```

```
depts=Button(ws, text='Departments', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_depts)
```

```
depts.place(x=150, y=430)
```

```
sports=Button(ws, text='Sports', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_sports)
```

```
sports.place(x=350, y=430)
```

```
events=Button(ws, text='Events', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=view_events)
```

```
events.place(x=550, y=430)
```

```
fees=Button(ws, text='Fees', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_fees)
fees.place(x=750, y=430)
```

```
labs=Button(ws, text='Labs', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_labs)
labs.place(x=950, y=430)
```

```
ws.mainloop()
```

Students

```
from tkinter import *
from tkinter import messagebox
import mysql.connector
```

```
ws = Tk()
ws.geometry('1300x700')
ws.title('Students-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)
```

```
def connect_database2():
    fname1 = fname.get()
    lname1 = lname.get()
    email1 = email.get()
    mobile1 = mobile.get()
    dob1 = dob.get()
    sclass1 = sclass.get()
    regno1 = regno.get()
    gender1 = vars.get()
    gen=""
    if gender1==1:
        gen="Male"
```

```

else:
    gen="Female"

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO student (FName, LName, Email, Mobile, DOB, Gender, SClass,
RegNo) VALUES (%s, %s, %s, %s, %s, %s, %s,%s)"
    val = (fname1,lname1,email1,mobile1,dob1,gen,sclass1,regno1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Student","Add Data Successfully")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_stud():
    import Viewstud

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_stud)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Student Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

heading=Label(ws, text='First Name:', font=('Arial', 20))
heading.place(x=200, y=200)

```

```
fname=Entry(ws, font=('Arial', 16), width=18)
fname.place(x=350,y=200)
```

```
heading=Label(ws, text='Last Name:', font=('Arial', 20))
heading.place(x=650, y=200)
```

```
lname=Entry(ws, font=('Arial', 16), width=18)
lname.place(x=800,y=200)
```

```
heading=Label(ws, text='EmailID:', font=('Arial', 20))
heading.place(x=200, y=300)
```

```
email=Entry(ws, font=('Arial', 16), width=18)
email.place(x=350,y=300)
heading=Label(ws, text='Mobile:', font=('Arial', 20))
heading.place(x=650, y=300)
```

```
mobile=Entry(ws, font=('Arial', 16), width=18)
mobile.place(x=800,y=300)
```

```
heading=Label(ws, text='DOB:', font=('Arial', 20))
heading.place(x=200, y=400)
```

```
dob=Entry(ws, font=('Arial', 16), width=18)
dob.place(x=350,y=400)
```

```
heading=Label(ws, text='Gender:', font=('Arial', 20))
heading.place(x=650, y=400)
```

```
#mobile2=Entry(ws, font=('Arial', 16), width=18)
```

```
vars = IntVar()
```

```
Radiobutton(ws, text="Male", font=('Arial', 16), padx=5,variable=vars, value=1).place(x=800,
y=400)
```

```

Radiobutton(ws, text="Female", font=('Arial', 16), padx =10,variable=vars,
value=2).place(x=900,y=400)
#Radiobutton(base, text="others", padx=15, variable=vars, value=3).place(x=310,y=240)

heading=Label(ws, text='Class:', font=('Arial', 20))
heading.place(x=200, y=500)

sclass=Entry(ws, font=('Arial', 16), width=18)
sclass.place(x=350,y=500)

heading=Label(ws, text='Reg No:', font=('Arial', 20))
heading.place(x=650, y=500)
regno=Entry(ws, font=('Arial', 16), width=18)
regno.place(x=800,y=500)

RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=12, height=1,
command=connect_database2)

RegButton.place(x=500, y=600)

ws.mainloop()

```

View students

```

import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo
FROM student")
    records = mycursor.fetchall()

```



```

print(records)
for i, (FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo) in enumerate(records,
start=1):
    listBox.insert("",                                "end",
values=(FName,LName,Email,Mobile,DOB,Gender,SClass,RegNo))
    mysqldb.close()
root = tk.Tk()
root.title("Student Records")
label = tk.Label(root, text="Student Records", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('FName','LName','Email','Mobile','DOB','Gender','SClass','RegNo')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Staffs

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Staffs-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    name1 = name.get()
    qual1 = qual.get()

```

```

email1 = email.get()
mobile1 = mobile.get()
salary1 = salary.get()
gender1 = vars.get()
gen=""
if gender1==1:
    gen="Male"
else:
    gen="Female"

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")

mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO staff (Name, Qualification, Email, Mobile, Salary, Gender) VALUES
(%s, %s, %s, %s, %s, %s)"
    val = (name1,qual1,email1,mobile1,salary1,gen)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Staff","Add Data Successfully")
except Exception as e:

    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_staff():
    import Viewstaff

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_staff)
ViewButton.place(x=1000, y=30)

```

```
heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Staff Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)
```

```
heading=Label(ws, text='Name:', font=('Arial', 20))
heading.place(x=200, y=200)
```

```
name=Entry(ws, font=('Arial', 16), width=18)
name.place(x=350,y=200)
```

```
heading=Label(ws, text='Qualification:', font=('Arial', 20))
heading.place(x=650, y=200)
```

```
qual=Entry(ws, font=('Arial', 16), width=18)
qual.place(x=800,y=200)
```

```
heading=Label(ws, text='EmailID:', font=('Arial', 20))
heading.place(x=200, y=300)
```

```
email=Entry(ws, font=('Arial', 16), width=18)
email.place(x=350,y=300)
```

```
heading=Label(ws, text='Mobile:', font=('Arial', 20))
heading.place(x=650, y=300)
```

```
mobile=Entry(ws, font=('Arial', 16), width=18)
mobile.place(x=800,y=300)
```

```
heading=Label(ws, text='Salary:', font=('Arial', 20))
heading.place(x=200, y=400)
salary=Entry(ws, font=('Arial', 16), width=18)
salary.place(x=350,y=400)
```

```

heading=Label(ws, text='Gender:', font=('Arial', 20))
heading.place(x=650, y=400)

#mobile2=Entry(ws, font=('Arial', 16), width=18)
vars = IntVar()
Radiobutton(ws, text="Male", font=('Arial', 16), padx=5, variable=vars, value=1).place(x=800,
y=400)
Radiobutton(ws, text="Female", font=('Arial', 16), padx=10, variable=vars,
value=2).place(x=900, y=400)
#Radiobutton(base, text="others", padx=15, variable=vars, value=3).place(x=310, y=240)

RegButton=Button(ws, text='Submit', font=('Arial', 16), bg='light grey', width=12, height=1,
command=connect_database2)
RegButton.place(x=500, y=500)
ws.mainloop()

```

View Staff

```

import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT Name,Qualification,Email,Mobile,Salary,Gender FROM
staff")
    records = mycursor.fetchall()
    print(records)
    for i, (Name,Qualification,Email,Mobile,Salary,Gender) in enumerate(records, start=1):
        listBox.insert("", "end", values=(Name,Qualification,Email,Mobile,Salary,Gender))
    mysqldb.close()
root = tk.Tk()
root.title("Staff Records")

```

```

label = tk.Label(root, text="Staff Records", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('Name','Qualification','Email','Mobile','Salary','Gender')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=6, column=1)
show()
root.mainloop()

```

Marks

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Marks-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    regno1 = regno.get()
    name1 = name.get()
    s11 = s1.get()
    s21 = s2.get()
    s31 = s3.get()
    s41 = s4.get()
    s51 = s5.get()
    s61 = s6.get()
    num1=int(s11)
    num2=int(s21)

```

```

num3=int(s31)
num4=int(s41)
num5=int(s51)
num6=int(s61)
t=num1+num2+num3+num4+num5+num6

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")

mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO marks (RegNo, Name, S1, S2, S3, S4, S5, S6, Total) VALUES (%s,
%s, %s, %s, %s, %s, %s, %s, %s)"
    val = (regno1,name1,s11,s21,s31,s41,s51,s61,t)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Marks","Add Data Successfully")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_mark():
    import Viewmark

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_mark)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Marks Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

```

```
heading=Label(ws, text='Reg No:', font=('Arial', 20))
heading.place(x=200, y=200)
```

```
regno=Entry(ws, font=('Arial', 16), width=18)
regno.place(x=350,y=200)
```

```
heading=Label(ws, text='Name:', font=('Arial', 20))
heading.place(x=650, y=200)
```

```
name=Entry(ws, font=('Arial', 16), width=18)
name.place(x=800,y=200)
```

```
heading=Label(ws, text='Subject 1:', font=('Arial', 20))
heading.place(x=200, y=300)
```

```
s1=Entry(ws, font=('Arial', 16), width=18)
s1.place(x=350,y=300)
```

```
heading=Label(ws, text='Subject 2:', font=('Arial', 20))
heading.place(x=650, y=300)
```

```
s2=Entry(ws, font=('Arial', 16), width=18)
s2.place(x=800,y=300)
```

```
heading=Label(ws, text='Subject 3:', font=('Arial', 20))
heading.place(x=200, y=400)
```

```
s3=Entry(ws, font=('Arial', 16), width=18)
s3.place(x=350,y=400)
```

```
heading=Label(ws, text='Subject 4:', font=('Arial', 20))
heading.place(x=650, y=400)
```

```
s4=Entry(ws, font=('Arial', 16), width=18)
```

```
s4.place(x=800,y=400)
```

```
heading=Label(ws, text='Subject 5:', font=('Arial', 20))
```

```
heading.place(x=200, y=500)
```

```
s5=Entry(ws, font=('Arial', 16), width=18)
```

```
s5.place(x=350,y=500)
```

```
heading=Label(ws, text='Subject 6', font=('Arial', 20))
```

```
heading.place(x=650, y=500)
```

```
s6=Entry(ws, font=('Arial', 16), width=18)
```

```
s6.place(x=800,y=500)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=12, height=1,  
command=connect_database2)
```

```
RegButton.place(x=500, y=600)
```

```
ws.mainloop()
```

View Marks

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",  
database="mydbs")
```

```
    mycursor = mysqldb.cursor()
```

```
    mycursor.execute("SELECT RegNo,Name,S1,S2,S3,S4,S5,S6,Total FROM marks")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```
    for i, (RegNo,Name,S1,S2,S3,S4,S5,S6,Total) in enumerate(records, start=1):
```

```
        listBox.insert("", "end", values=(RegNo,Name,S1,S2,S3,S4,S5,S6,Total))
```



```

mysqldb.close()

root = tk.Tk()
root.title("Mark Details")
label = tk.Label(root, text="View Mark Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('RegNo','Name','S1','S2','S3','S4','S5','S6','Total')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Subjects

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Subjects-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    subject1 = subject.get()
    dept1 = dept.get()
    staff1 = staff.get()
    no=0

```

```

sno=no
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO subjects (SNo, Subject, Department, Staff) VALUES (%s, %s, %s,
%s)"
    val = (sno,subject1,dept1,staff1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Subjects", "Subject is Added")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()
def view_subject():
    import Viewssubject

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_subject)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Subjects Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

heading=Label(ws, text='Subject:', font=('Arial', 20))
heading.place(x=350, y=200)

subject=Entry(ws, font=('Arial', 16), width=18)
subject.place(x=550,y=200)

```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
```

```
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
```

```
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Staff:', font=('Arial', 20))
```

```
heading.place(x=350, y=400)
```

```
staff=Entry(ws, font=('Arial', 16), width=18)
```

```
staff.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,  
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
```

```
ws.mainloop()
```

View Subjects

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",  
database="mydbs")
```

```
    mycursor = mysqldb.cursor()
```

```
    mycursor.execute("SELECT SNo,Subject,Department,Staff FROM subjects")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```
    for i, (SNo,Subject,Department,Staff) in enumerate(records, start=1):
```

```
        listBox.insert("", "end", values=(SNo,Subject,Department,Staff))
```

```
    mysqldb.close()
```

```
root = tk.Tk()
```

```
root.title("Subject Records")
```

```

label = tk.Label(root, text="View Subject Details", font=("Arial",24)).grid(row=0,
columnspan=3)
cols = ('SNo','Subject','Department','Staff')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Exams

```

from tkinter import *
from tkinter import messagebox
import mysql.connector
from tkcalendar import Calendar, DateEntry

```

```

ws = Tk()
ws.geometry('1300x700')
ws.title('Exams-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

```

```

def connect_database2():

```

```

    ed1 = ed.get()
    dept1 = dept.get()
    subject1 = subject.get()
    sem1 = clicked.get()

```

```

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

```

```

try:
    sql = "INSERT INTO exams (EDate, Department, Subject, Semester) VALUES (%s, %s, %s, %s)"
    val = (ed1,dept1,subject1,sem1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Exam","Success")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()
options = [
    "1",
    "2",
    "3",
    "4",
    "5",
    "6",
]
clicked = StringVar()
clicked.set( "Select Sem" )

def view_exam():
    import Viewexam

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_exam)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Exam Date & Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

```

```
heading=Label(ws, text='Date:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
ed = DateEntry(ws, font=('Arial', 16), width=18)
ed.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=300)
```

```
heading=Label(ws, text='Subject:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
subject=Entry(ws, font=('Arial', 16), width=18)
subject.place(x=550,y=400)
```

```
heading=Label(ws, text='Semester:', font=('Arial', 20))
heading.place(x=350, y=500)
```

```
#subject=Entry(ws, font=('Arial', 16), width=18)
sem = OptionMenu(ws , clicked , *options)
sem.place(x=550,y=500)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
RegButton.place(x=550, y=550)
ws.mainloop()
```

View Exams

```
import tkinter as tk
from tkinter import ttk
import mysql.connector
```

```

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT EDate,Department,Subject,Semester FROM exams")
    records = mycursor.fetchall()
    print(records)
    for i, (EDate,Department,Subject,Semester) in enumerate(records, start=1):
        listBox.insert("", "end", values=(EDate,Department,Subject,Semester))
    mysqldb.close()
root = tk.Tk()
root.title("Exam Records")
label = tk.Label(root, text="View Exam Details", font=("Arial",24)).grid(row=0,
columnspan=3)
cols = ('EDate','Department','Subject','Semester')
listBox = ttk.Treeview(root, columns=cols, show='headings')
for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Department

```

from tkinter import *
from tkinter import messagebox
import mysql.connector
ws = Tk()
ws.geometry('1300x700')
ws.title('Department-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():

```

```

degree1 = degree.get()
dept1 = dept.get()
desc1 = desc.get("1.0", "end-1c")
no=0
sno=no

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()
try:
    sql = "INSERT INTO depts (SNo, Degree, Department, Description) VALUES (%s, %s,
%s, %s)"
    val = (sno,degree1,dept1,desc1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Department","Success")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_dept():
    import Viewdept
    ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_dept)
    ViewButton.place(x=1000, y=30)
    heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
    heading.place(x=430, y=70)
    heading : Label = Label(ws, text='Department Details', font=('Arial', 20), fg='red')
    heading.place(x=550, y=120)
    heading=Label(ws, text='Degree:', font=('Arial', 20))
    heading.place(x=350, y=200)

degree=Entry(ws, font=('Arial', 16), width=18)

```



```

degree.place(x=550,y=200)
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=300)

dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=300)

heading=Label(ws, text='Description:', font=('Arial', 20))
heading.place(x=350, y=400)

desc=Text(ws, height = 5, width = 52)
desc.place(x=550,y=400)

RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)

RegButton.place(x=550, y=500)

ws.mainloop()

import tkinter as tk
from tkinter import ttk
import mysql.connector

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT SNo,Degree,Department,Description FROM depts")
    records = mycursor.fetchall()
    print(records)

    for i, (SNo,Degree,Department,Description) in enumerate(records, start=1):

```

```

        listBox.insert("", "end", values=(SNo,Degree,Department,Description))
    mysqldb.close()

root = tk.Tk()
root.title("Department Details")
label = tk.Label(root, text="View Exam Details", font=("Arial",24)).grid(row=0,
columnspan=3)

cols = ('SNo','Degree','Department','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Sports

```

from tkinter import *
from tkinter import messagebox
import mysql.connector
ws = Tk()
ws.geometry('1300x700')
ws.title('Sports-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)
def connect_database2():
    sports1 = sports.get()
    age1 = age.get()
    height1=height.get()
    weight1=weight.get()
    desc1 = desc.get("1.0", "end-1c")
    no=0

```

```

sno=no

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")

mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO sports (Sports, Age, Height, Weight, Description) VALUES (%s, %s,
%s, %s, %s)"
    val = (sports1,age1,height1,weight1,desc1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Sports","Success")
except Exception as e:
    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_sport():
    import Viewsport
    ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_sport)
    ViewButton.place(x=1000, y=30)
    heading : Label = Label(ws, text='College Management System', font=('Arial', 24), fg='#8e44ad')
    heading.place(x=430, y=70)
    heading : Label = Label(ws, text='Sports Details', font=('Arial', 20), fg='red')
    heading.place(x=550, y=120)

    heading=Label(ws, text='Sports:', font=('Arial', 20))
    heading.place(x=350, y=200)

    sports=Entry(ws, font=('Arial', 16), width=18)
    sports.place(x=550,y=200)

    heading=Label(ws, text='Age:', font=('Arial', 20))

```

```
heading.place(x=350, y=250)
```

```
age=Entry(ws, font=('Arial', 16), width=18)
```

```
age.place(x=550,y=250)
```

```
heading=Label(ws, text='Hight:', font=('Arial', 20))
```

```
heading.place(x=350, y=300)
```

```
height=Entry(ws, font=('Arial', 16), width=18)
```

```
height.place(x=550,y=300)
```

```
heading=Label(ws, text='Weight:', font=('Arial', 20))
```

```
heading.place(x=350, y=350)
```

```
weight=Entry(ws, font=('Arial', 16), width=18)
```

```
weight.place(x=550,y=350)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
```

```
heading.place(x=350, y=400)
```

```
desc=Text(ws, height = 5, width = 52)
```

```
desc.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,  
command=connect_database2)
```

```
RegButton.place(x=550, y=500)
```

```
ws.mainloop()
```

View Sports

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```

mysqlldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
mycursor = mysqlldb.cursor()
mycursor.execute("SELECT Sports,Age,Height,Weight,Description FROM sports")
records = mycursor.fetchall()
print(records)
for i, (Sports,Age,Height,Weight,Description) in enumerate(records, start=1):
    listBox.insert("", "end", values=(Sports,Age,Height,Weight,Description))
mysqlldb.close()

root = tk.Tk()
root.title("Sports Records")
label = tk.Label(root, text="View Sports Details", font=("Arial",24)).grid(row=0,
columnspan=3)
cols = ('Sports','Age','Height','Weight','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')
for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7, column=1)
show()
root.mainloop()

```

Events

```

from tkinter import *
from tkinter import messagebox
import mysql.connector
from tkcalendar import Calendar, DateEntry
ws = Tk()
ws.geometry('1300x700')
ws.title('Events-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():

```

```

ed1 = ed.get()
en1 = en.get()
place1 = place.get()
desc1 = desc.get("1.0", "end-1c")
mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO events (EDate, Event, Place, Description) VALUES (%s, %s, %s,
%s)"
    val = (ed1,en1,place1,desc1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Events","Success")
except Exception as e:

    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_event():
    import Viewevent

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_event)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24),
fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Event Date & Place', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

```

```
heading=Label(ws, text='Date:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
ed = DateEntry(ws, font=('Arial', 16), width=18)
ed.place(x=550,y=200)
```

```
heading=Label(ws, text='EventName:', font=('Arial', 20))
heading.place(x=350, y=250)
```

```
en=Entry(ws, font=('Arial', 16), width=18)
en.place(x=550,y=250)
```

```
heading=Label(ws, text='Place:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
place=Entry(ws, font=('Arial', 16), width=18)
place.place(x=550,y=300)
```

```
heading=Label(ws, text='Description:', font=('Arial', 20))
heading.place(x=350, y=350)
```

```
desc=Text(ws, height = 5, width = 52)
desc.place(x=550,y=350)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
ws.mainloop()
```

View Events

```
import tkinter as tk
from tkinter import ttk
import mysql.connector
```

```

def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
    database="mydbs")
    mycursor = mysqldb.cursor()
    mycursor.execute("SELECT EDate,Event,Place,Description FROM events")
    records = mycursor.fetchall()
    print(records)

    for i, (EDate,Event,Place,Description) in enumerate(records, start=1):
        listBox.insert("", "end", values=(EDate,Event,Place,Description))
        mysqldb.close()

root = tk.Tk()
root.title("Event Records")
label = tk.Label(root, text="View Event Details", font=("Arial",24)).grid(row=0,
columnspan=3)

cols = ('EDate','Event','Place','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7,
column=1)
show()
root.mainloop()

```

Fees

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

```



```

ws = Tk()
ws.geometry('1300x700')
ws.title('Fees Details-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    degree1 = degree.get()
    dept1 = dept.get()
    tfee1 = tfee.get()
    lfee1 = lfee.get()
    sfee1 = sfee.get()
    tf = int(tfee1)
    lf = int(lfee1)
    sf = int(sfee1)
    t = tf+lf+sf
    no=0
    sno=no

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO fees (SNo, Degree, Department, TutionFee, LabFee, SFee, Total)
VALUES (%s, %s, %s, %s, %s, %s, %s)"
    val = (sno,degree1,dept1,tfee1,lfee1,sfee1,t)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Fees","Success")
except Exception as e:

    print(e)

```

```

mysqlodb.rollback()
mysqlodb.close()

def view_fee():
    import Viewfee

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_fee)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24),
fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Fee Details / Year', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

heading=Label(ws, text='Degree:', font=('Arial', 20))
heading.place(x=350, y=200)

degree=Entry(ws, font=('Arial', 16), width=18)
degree.place(x=550,y=200)
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=250)

dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=250)

heading=Label(ws, text='TutionFee:', font=('Arial', 20))
heading.place(x=350, y=300)

tfee=Entry(ws, font=('Arial', 16), width=18)
tfee.place(x=550,y=300)

```

```
heading=Label(ws, text='LabFee:', font=('Arial', 20))
heading.place(x=350, y=350)
```

```
lfee=Entry(ws, font=('Arial', 16), width=18)
lfee.place(x=550,y=350)
```

```
heading=Label(ws, text='Stationary:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
sfee=Entry(ws, font=('Arial', 16), width=18)
sfee.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=450)
```

```
ws.mainloop()
```

View Fees Details

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import mysql.connector
```

```
def show():
```

```
    mysqlldb = mysql.connector.connect(host="localhost", user="root", password="good",
database="mydbs")
```

```
    mycursor = mysqlldb.cursor()
```

```
    mycursor.execute("SELECT SNo,Degree,Department,TutionFee,LabFee,SFee,Total
FROM fees")
```

```
    records = mycursor.fetchall()
```

```
    print(records)
```

```

        for i, (SNo,Degree,Department,TutionFee,LabFee,SFee>Total) in enumerate(records,
start=1):
            listBox.insert("", "end",
values=(SNo,Degree,Department,TutionFee,LabFee,SFee>Total))
            mysqlodb.close()

root = tk.Tk()
root.title("Fees Details")
label = tk.Label(root, text="View Fee Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('SNo','Degree','Department','TutionFee','LabFee','SFee','Total')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7,
column=1)
show()
root.mainloop()

```

Labs

```

from tkinter import *
from tkinter import messagebox
import mysql.connector

ws = Tk()
ws.geometry('1300x700')
ws.title('Laboratory-College Management System')
ws['bg']='#ecf0f1'
f = ("Times bold", 14)

def connect_database2():
    ln1 = ln.get()

```

```

dept1 = dept.get()
desc1 = desc.get("1.0", "end-1c")
no=0
sno=no

mysqldb=mysql.connector.connect(host="localhost",user="root",password="good",database="
mydbs")
mycursor=mysqldb.cursor()

try:
    sql = "INSERT INTO labs (SNo, LabName, Department, Description) VALUES (%s, %s,
%s, %s)"
    val = (sno,ln1,dept1,desc1)
    mycursor.execute(sql, val)
    mysqldb.commit()
    messagebox.showinfo("Laboratory","Add Lab Details")
except Exception as e:

    print(e)
    mysqldb.rollback()
    mysqldb.close()

def view_lab():
    import Viewlab

ViewButton=Button(ws, text='View', font=('Arial', 16),bg='light grey', width=12, height=1,
command=view_lab)
ViewButton.place(x=1000, y=30)

heading : Label = Label(ws, text='College Management System', font=('Arial', 24),
fg='#8e44ad')
heading.place(x=430, y=70)
heading : Label = Label(ws, text='Laboratory-Details', font=('Arial', 20), fg='red')
heading.place(x=550, y=120)

```

```
heading=Label(ws, text='LabName:', font=('Arial', 20))
heading.place(x=350, y=200)
```

```
ln=Entry(ws, font=('Arial', 16), width=18)
ln.place(x=550,y=200)
```

```
heading=Label(ws, text='Department:', font=('Arial', 20))
heading.place(x=350, y=300)
```

```
dept=Entry(ws, font=('Arial', 16), width=18)
dept.place(x=550,y=300)
heading=Label(ws, text='Description:', font=('Arial', 20))
heading.place(x=350, y=400)
```

```
desc=Text(ws, height = 5, width = 52)
desc.place(x=550,y=400)
```

```
RegButton=Button(ws, text='Submit', font=('Arial', 16),bg='light grey', width=18,
command=connect_database2)
```

```
RegButton.place(x=550, y=500)
```

```
ws.mainloop()
```

View Labs

```
import tkinter as tk
from tkinter import ttk
import mysql.connector
```

```
def show():
    mysqldb = mysql.connector.connect(host="localhost", user="root", password="good",
    database="mydbs")
    mycursor = mysqldb.cursor()
```

```

mycursor.execute("SELECT SNo,LabName,Department,Description FROM labs")
records = mycursor.fetchall()
print(records)

for i, (SNo,LabName,Department,Description) in enumerate(records, start=1):
    listBox.insert("", "end", values=(SNo,LabName,Department,Description))
mysqldb.close()

root = tk.Tk()
root.title("Lab Details")
label = tk.Label(root, text="View Fee Details", font=("Arial",24)).grid(row=0, columnspan=3)

cols = ('SNo','LabName','Department','Description')
listBox = ttk.Treeview(root, columns=cols, show='headings')

for col in cols:
    listBox.heading(col, text=col)
    listBox.grid(row=1, column=0, columnspan=1)
#closeButton = tk.Button(root, text="Close", width=15, command=exit).grid(row=7,
column=1)
show()
root.mainloop()

```

8.REPORTS

STUDENT:

College Management System
Student Details

View

FName	LName	Email	Mobile	DOB	Gender	SClass
Ravi	S	ravi@gmail.com	9876543210	30-12-1993	Male	BSc
Siva	D	siva@gmail.com	9876543210	22-10-1999	Female	CSE
Samir	A	shamir@gmail.com	9876543210	10-12-2007	Female	BE-CSE

Class: Reg No:

Submit

STAFF:

Staff Records

Name	Qualification	Email	Mobile	Salary	Gender
Rajiv	PhD	Rajiv@gmail.com	9876543210	120000	Male

Salary: Gender: ☐ Male ☒ Female

Submit

MARK:

College Management System Using Python and MySQL - Microsoft Word

Marks-College Management System

View

Mark Details

View Mark Details

RegNo	Name	S1	S2	S3	S4	S5
10503565	Siva	78	56	34	87	87

Subject 3: Subject 4:

Subject 5: Subject 6:

Submit

Page: 82

4:59 PM 28-Dec-24

SUBJECT:

College Management System Using Python and MySQL - Microsoft Word

Subjects-College Management System

View

College Management System

Subject Records

View Subject Details

SNo	Subject	Department	Staff
1	TOC	CSE	Rajiv
2	C#	CSE	Rajiv

Submit

Page: 83

4:59 PM 28-Dec-24

EXAM:

College Management System Using Python and MySQL - Microsoft Word

Exam Records

View Exam Details

EDate	Department	Subject	Semester
1/2/25	CSE	TOC	3

View

Department:

Subject:

Semester:

Submit

DEPARTMENT:

DashBoard-College Management System

Department Details

View Department Details

SNo	Degree	Department	Description
4	BE	CSE	Computer Science

View

Department:

Description:

Submit

SPORTS:

College Management System Using Python and MySQL - Microsoft Word

Sports-College Management System

Sports Records

View Sports Details

Sports	Age	Height	Weight	Description
Cricket	17	6	50	All Clear

View

Hight:

Weight:

Description:

Submit

Page: 87

5:02 PM 28-Dec-24

EVENT:

College Management System Using Python and MySQL - Microsoft Word

Events-College Management System

Event Records

View Event Details

EDate	Event	Place	Description
1/5/25	Workshop	Lab 2	For All IT Students

View

Description:

Submit

Page: 87

5:03 PM 28-Dec-24

FEE:

College Management System Using Python and MySQL - Microsoft Word

Fees Details-College Management System

View

College Management System

Fees Details

View Fee Details

SNo	Degree	Department	TutionFee	LabFee	SFee	Total
2	BE	CSE	20000	6000	2500	28500

Stationary:

Submit

Page: 88

5:03 PM
28-Dec-24

9. CONCLUSION

The college management system is an application to provide an integrated inquiry regarding students and mainly for staffs who work in the college. This application software was module successfully and was also tested successfully by running test cases. It is easy to use and has the required options, which the user can use to perform the required operations.

FUTURE SCOPE

Future enhancements for the project College Management System using web applications can focus on improving various aspects of the system to make it more efficient, user-friendly, and feature-rich. Here are some potential enhancements like Alumni Networking, Gamification Elements, Virtual Classrooms and many more.

10. BIBLIOGRAPHY

- <https://docs.python.org/>
- <https://docs.python.org/>
- <https://dev.mysql.com/doc/>](<https://dev.mysql.com/doc/>)
- <https://realpython.com/python-mysql/>](<https://realpython.com/python-mysql/>)
- <https://pythonprogramming.net/>